

P2: Creating and Working with Your Database

Group 22: Gonzalo Millán Garcia, Annabelle Huynh-Rondeau, Jessie Qiao, Clementine Ruiz

Updated Relational Schema based on feedback

Artifacts(artifact_id, name, description, date_of_creation)

Paintings(artifact_id, medium, paint_type) foreign key artifact_id references Artifacts

Photographs(artifact_id, film_type) foreign key artifact_id references Artifacts

Sculpture(artifact_id, material, dimensions) foreign key artifact_id references Artifacts

Exhibitions(exhibition_id, name, description, start_date, end_date)

Displays(artifact_id, exhibition_id, room) foreign key artifact_id references Artifacts and foreign key exhibition_id references Exhibitions

Artists(artist_id, name, birth)

Creates(artifact_id, artist_id) foreign key artifact_id references Artifacts and foreign key artist_id references Artists

Genres(genre_name)

Exhibition_Genre(exhibition_id, genre_name) foreign key exhibition_id references Exhibitions and foreign key genre_name references Genres

Artifact_Genre(artifact_id, genre_name) foreign key artifact_id references Artifacts and foreign key genre_name references Genres

Artifact_Genre(artifact_id, genre_name) foreign key artifact_id references Artifacts and foreign key genre_name references Genres

Tours(tour_id, description, cost, employee_id) foreign key employee_id references Museum_Personnel

Museum_Personnel

Tour_Exhibitions(exhibition_id, tour_id) foreign key exhibition_id references Exhibition and foreign key tour_id references Tours

Museum_Personnel(employee_id, name)

Organizer(employee_id, tour_id) foreign key employee_id references Museum_Personnel and foreign key tour_id references Tours

TourDates(date, start_time, tour_id, end_time) foreign key tour_id references Tours

Visitor(email, name, address, birth)

Reservation(email, booking_date, tour_id, date, start_time) foreign key email references Visitor, foreign keys date, start_time references TourDates, tour_id references Tours

Visitor, foreign keys date, start_time references TourDates, tour_id references Tours

Pending constraints

That we cannot enforce through the CREATE SQL queries.

- 1) Staff Overlap: We cannot express the constraint that a guide cannot lead overlapping tours.
- 2) Tour capacity: Maximum capacity for tour reservation must be handled by the application layer.
- 3) Booking window: Ensuring that the booking date precedes the tour date involves cross-table validation.
- 4) Mandatory Genre Participation: Every artifact must belong to at least one genre, which is a total participation constraint.

SQL queries

Query 1

Description: A query that selects all the artifacts that are in storage (not displayed in any exhibition), and the artists that made each artifact. Tables: Artifacts, Display, Artists, Creates.

SQL:

```
WITH InStorage AS (  
    SELECT  
        A.artifact_id,  
        A.name,  
        A.description,  
        A.date_of_creation  
    FROM Artifacts A  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM Displays D  
        WHERE D.artifact_id = A.artifact_id  
    )  
)  
SELECT  
    S.artifact_id,  
    S.name AS artifact_name,  
    S.description,  
    S.date_of_creation,  
    AR.artist_id,  
    AR.name AS artist_name,  
    AR.birth AS artist_birth  
FROM InStorage S  
JOIN Creates C  
    ON C.artifact_id = S.artifact_id  
JOIN Artists AR  
    ON AR.artist_id = C.artist_id  
ORDER BY S.name, AR.name;
```

Screenshot:

```
cs42ig22@winter2026-comp421:~$ db2 "WITH InStorage AS (SELECT A.artifact_id, A.name, A.description, A.date_of_creation FROM Artifacts A WHERE NOT EXISTS (SELECT 1 FROM Displays D WHERE D.artifact_id = A.artifact_id)) SELECT S.artifact_id, S.name AS artifact_name, S.description, S.date_of_creation, AR.artist_id, AR.name AS artist_name, AR.birth AS artist_birth FROM InStorage S JOIN Creates C ON C.artifact_id = S.artifact_id JOIN Artists AR ON AR.artist_id = C.artist_id ORDER BY S.name, AR.name"
```

ARTIFACT_ID	ARTIFACT_NAME	DESCRIPTION	DATE_OF_CREATION	ARTIST_ID	ARTIST_NAME	ARTIST_BIRTH
46	A Lesson in Humility	A Lesson in Humility depicts a famous ancient athlete, Milo, being trapped and devoured by a lion. The sculpture symbolizes the fall of human strength and pride against nature. It represents the height of French Baroque sculpture, characterized by intense emotion, muscular tension, and dramatic motion. - by Pierre Puget				
67	Casque orne d'une tete de loup	Headpiece in bronze. - by Martin van den Bogaert	1671	24	Pierre Puget	18/16/1620
10	Ceres	The statue depicts Ceres, the Roman goddess of agriculture, fertility, and summer. Standing over three meters tall, the figure is richly detailed in cascading drapery and classical form, reflecting the influence of antiquity on 18th century French sculpture. - by Guillaume Coustou	1685	35	Martin van den Bogaert	-
50	Christ on the Cross		1731	8	Guillaume Coustou	11/29/1677

Query 2

Description: A query that ranks exhibitions by diversity, by finding the aggregate number of distinct genres, grouping by exhibition and ordering by this “diversity value”. Tables: Exhibitions, Display, Artifact_Genre.

SQL:

```
SELECT
    e.exhibition_id,
    e.name,
    COUNT(DISTINCT ag.genre_name) AS diversity_value
FROM Exhibitions e
JOIN Displays d
    ON d.exhibition_id = e.exhibition_id
JOIN Artifact_Genre ag
    ON ag.artifact_id = d.artifact_id
GROUP BY e.exhibition_id, e.name
ORDER BY diversity_value DESC, e.name;
```

Screenshot:

```
db2 => SELECT e.exhibition_id, e.name, COUNT(DISTINCT ag.genre_name) AS diversity_value
db2 (cont.) => FROM Exhibitions e
db2 (cont.) => JOIN Displays d ON d.exhibition_id = e.exhibition_id
db2 (cont.) => JOIN Artifact_Genre ag ON ag.artifact_id = d.artifact_id
db2 (cont.) => GROUP BY e.exhibition_id, e.name
db2 (cont.) => ORDER BY diversity_value DESC, e.name;
```

EXHIBITION_ID	NAME	DIVERSITY_VALUE
6	Textiles and tapestry	6
3	Portraits	4
8	exhibition01	4
2	Renaissance masters	2
9	exhibition02	2
1	Ancient civilizations	1
4	Arms and armor	1
5	Japanese art	1
7	The age of romanticism	1
10	exhibition03	1

Query 3

Description: A query that gets the top 10 most collaborative artists by number of artifacts that were created in collaboration with another artist. Tables: Artists, Creates.

SQL:

```
WITH CollaborativeArtifacts AS (
    SELECT c.artifact_id
    FROM Creates c
    GROUP BY c.artifact_id
    HAVING COUNT(DISTINCT c.artist_id) >= 2
),
ArtistCollabCounts AS (
    SELECT c.artist_id,
    COUNT(DISTINCT c.artifact_id) AS collaborative_artifacts
    FROM Creates c
    JOIN CollaborativeArtifacts ca
        ON ca.artifact_id = c.artifact_id
    GROUP BY c.artist_id
)
SELECT
    ar.artist_id,
    ar.name,
    acc.collaborative_artifacts
FROM ArtistCollabCounts acc
JOIN Artists ar
    ON ar.artist_id = acc.artist_id
```

```
ORDER BY acc.collaborative_artifacts DESC, ar.name
FETCH FIRST 10 ROWS ONLY;
```

Screenshot:

```
db2 => WITH CollaborativeArtifacts AS (SELECT c.artifact_id FROM Creates c GROUP BY c.artifact_id HAVING COUNT(DISTINCT c.artist_id) >= 2), ArtistCollabCounts AS (SELECT c.artist_id, COUNT(DISTINCT c.artifact_id) AS collaborative_artifacts FROM Creates c JOIN CollaborativeArtifacts ca ON ca.artifact_id = c.artifact_id GROUP BY c.artist_id) SELECT ar.artist_id, ar.name, acc.collaborative_artifacts FROM ArtistCollabCounts acc JOIN Artists ar ON ar.artist_id = acc.artist_id ORDER BY acc.collaborative_artifacts DESC, ar.name FETCH FIRST 10 ROWS ONLY;
```

ARTIST_ID	NAME	COLLABORATIVE_ARTIFACTS
43	Alexandre-Louis Bellange	1
41	Amy Huaud	1
27	Evrard Bapst	1
49	Franz Xavier Winterhalter	1
28	Frederic Bapst	1
48	Gustave Adolphe Chassevent	1
44	Jean Jacques Feuchere	1
42	Jean-Pierre Huaud	1
40	Julien Le Roy	1

9 record(s) selected.

Query 4

Description: A query that ranks the tours based on popularity by aggregating the amount of bookings made, used for demand analytics. Tables: Tours, Reservation.

SQL:

```
SELECT
    t.tour_id,
    t.description,
    COUNT(r.email) AS booking_count
FROM Tours t
LEFT JOIN Reservation r
    ON t.tour_id = r.tour_id
GROUP BY t.tour_id, t.description
ORDER BY booking_count DESC;
```

Screenshot:

```
cs421g22@winter2026-comp421:~$ db2 "SELECT t.tour_id, t.description, COUNT(r.email) AS booking_count FROM Tours t LEFT JOIN Reservation r ON t.tour_id = r.tour_id GROUP BY t.tour_id, t.description ORDER BY booking_count DESC"
```

TOUR_ID	DESCRIPTION	BOOKING_COUNT
4	Tour explores ancient Egyptian and Mesopotamian artifacts.	9
2	Romanticism and revolutionary art tour.	6
1	Highlights of the Louvre including famous masterpieces.	4
5	Renaissance art tour featuring Leonardo da Vinci and Michelangelo.	

Query 5

Description: A query that returns the list of available tours, including information about them. Intended to be used by users. Tables: Tours, TourDates, Tour_Exhibitions, Exhibitions.

SQL:

```
SELECT
    t.tour_id,
    t.description,
    t.cost,
    td.date,
    td.start_time,
    td.end_time,
    e.name AS exhibition_name
FROM Tours t
JOIN TourDates td
    ON t.tour_id = td.tour_id
JOIN Tour_Exhibitions te
    ON t.tour_id = te.tour_id
JOIN Exhibitions e
    ON te.exhibition_id = e.exhibition_id
WHERE td.date >= CURRENT DATE
ORDER BY td.date, td.start_time;
```

Screenshot:

```
db2 => SELECT t.tour_id, t.description, t.cost, td.date, td.start_time, td.end_time, e.name AS exhibition_name
db2 (cont.) => FROM Tours t
db2 (cont.) => JOIN TourDates td ON t.tour_id = td.tour_id
db2 (cont.) => JOIN Tour_Exhibitions te ON t.tour_id = te.tour_id
db2 (cont.) => JOIN Exhibitions e ON te.exhibition_id = e.exhibition_id
db2 (cont.) => WHERE td.date >= CURRENT DATE
db2 (cont.) => ORDER BY td.date, td.start_time;

TOUR_ID      DESCRIPTION
-----
5 Renaissance art tour featuring Leonardo da Vinci and Michelangelo.

COST      DATE      START_TIME  END_TIME  EXHIBITION_NAME
-----
7.50 04/10/2026 10:00:00  11:00:00  exhibition02
```

5 Renaissance art tour featuring Leonardo da Vinci and Michelangelo.	7.50	04/10/2026 12:00:00	13:00:00	exhibition02
5 Renaissance art tour featuring Leonardo da Vinci and Michelangelo.				
4 Tour explores ancient Egyptian and Mesopotamian artifacts.	7.50	04/10/2026 14:00:00	15:00:00	exhibition02
4 Tour explores ancient Egyptian and Mesopotamian artifacts.				
4 Tour explores ancient Egyptian and Mesopotamian artifacts.	12.00	04/15/2026 10:00:00	11:30:00	Ancient civilizations
4 Tour explores ancient Egyptian and Mesopotamian artifacts.				

SQL Modifications

Mod 1

Description: A data modification command which deletes all artifacts that were created before year 0. This deletes 6 entries in our database.

SQL:

```
DELETE FROM Artifacts
WHERE date_of_creation < 0;
```

Screenshot:

```
[db2 => DELETE FROM Artifacts WHERE date_of_creation < 0;
DB20000I The SQL command completed successfully.
```

Mod 2

Description: A data modification command which adds the name of the artist to the description of each artifact. Tables: Artifacts, Creates, Artists

SQL:

```
UPDATE Artifacts art
SET description = description || ' - by ' || (
    SELECT LISTAGG(a.name, ', ')
    FROM Artists a, Creates c
    WHERE c.artist_id = a.artist_id
    AND c.artifact_id = art.artifact_id
);
```

** Each artifact can be associated with multiple artists. Here LISTAGG allows us to aggregate all artists involved in the creation of an artefact.

Screenshot:

```
[db2 => UPDATE Artifacts art SET description = description || ' - by ' || (
      SELECT LISTAGG(a.name, ', ')      FROM Artists a, Creates c      WHERE c
      .artist_id = a.artist_id      AND c.artifact_id = art.artifact_id );
DB20000I  The SQL command completed successfully.
```

Views

View 1

Description: A view which contains the name and date of creation of all artifacts created during or after the year 2000. Entries with NULL instead of a creation date are excluded from this view.

SQL:

```
CREATE VIEW recent_artifacts
AS
SELECT name, date_of_creation
FROM Artifacts
WHERE date_of_creation >= 2000;
```

Statement being executed in DB2:

```
[db2 => CREATE VIEW recent_artifacts
[db2 (cont.) => AS
[db2 (cont.) => SELECT name, date_of_creation
[db2 (cont.) => FROM Artifacts
[db2 (cont.) => WHERE date_of_creation >= 2000;
DB20000I  The SQL command completed successfully.
```

Screenshot of first 5 records

```
db2 => SELECT * from recent_artifacts FETCH FIRST 5 ROWS ONLY;
```

NAME	DATE_OF_CREATION
Ni apparence ni illusion. "Le mur est la surface de sa propre histoire enfouie. Tant que les objets restent dessus, le spectateur cache la vue a l'archologue."	2005
Ni apparence ni illusion. "Quinze pierres en place, toutes sorties de l'ombre, ces mots lumineux rendent visibles a la fois celui qui voit et celui qui est vu. Le mur, le passage."	2005
Ni apparence ni illusion. "detail se dilate."	2005

3 record(s) selected.

Screenshot for insert:

```
[db2 => INSERT INTO recent_artifacts VALUES ('Test', 2020);
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0407N  Assignment of a NULL value to a NOT NULL column "TBSPACEID=2,
TABLEID=17936, COLNO=0" is not allowed.  SQLSTATE=23502
```

Explanation on results of attempted insert:

The attempted insert failed as we have not specified a value for the artifact_id column. Indeed, this column is a primary key for the base table (Artifacts) from which the view is selecting rows and cannot be null. As we did not specify a value, when we created the entry, artifact_id was set to null by default.

View 2

Description: A view which contains the id for every tour and the associated number of reservations

SQL:

```
CREATE VIEW tour_stats
AS
SELECT T.tour_id, COUNT(*) AS number_of_reservations
FROM Tours T
LEFT JOIN Reservation R
ON T.tour_id=R.tour_id
GROUP BY T.tour_id;
```

Statement being executed in DB2:

```
db2 => CREATE VIEW tour_stats AS SELECT T.tour_id, COUNT(*) AS number_of_reservations FROM Tours T LEFT JOIN Reservation R ON T.tour_id=R.tour_id GROUP BY T.tour_id;
DB20000I The SQL command completed successfully.
```

Screenshot of a SQL query:

```
[db2 => SELECT * FROM tour_stats FETCH FIRST 5 ROWS ONLY;
```

TOUR_ID	NUMBER_OF_RESERVATIONS
1	4
2	6
3	3
4	9
5	4

5 record(s) selected.

Screenshot for insert:

```
[db2 => INSERT INTO tour_stats VALUES (501, 1);
DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0150N The target fullselect, view, typed table, materialized query table
,
range-clustered table, or staging table in the INSERT, DELETE, UPDATE, MERGE
,
or TRUNCATE statement is a target for which the requested operation is not
permitted. SQLSTATE=42807]
```

Explanation on results of attempted insert:

The insert failed as the view contains aggregation and a GROUP BY clause. Indeed, as the rows in the view do not directly correspond to rows in any base table, DB2 cannot determine how to perform the insertion and therefore does not allow it.

Check constraints

Check 1

What the constraint achieves: This constraint avoids exhibitions that have an end date before their start date in the Exhibitions table.

Constraint statement:

```
ALTER TABLE Exhibitions
ADD CONSTRAINT check_exhibition_dates
CHECK (end_date >= start_date);
```

Screenshot:

```
[db2 => ALTER TABLE Exhibitions ADD CONSTRAINT check_exhibition_dates CHECK (end_
date >= start_date);
DB20000I The SQL command completed successfully.]
```

Screenshot of attempt to insert violating record:

```
db2 => INSERT INTO Exhibitions (exhibition_id, name, description, start_date, en
d_date)
VALUES (99, 'Invalid Exhibition', 'Test constraint', DATE('2026-08-01'), DATE('2
026-05-01'));
db2 (cont.) => DB21034E The command was processed as an SQL statement because i
t was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0545N The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G22.EXHIBITIONS.CHECK_EXHIBITION_DATES".
SQLSTATE=23513
```

Check 2

What the constraint achieves: This constraint prevents costs that are negative and remain within a reasonable range for museum tours to avoid nonsensical pricing of Tours.

Constraint statement:

```
ALTER TABLE Tours
ADD CONSTRAINT check_tour_cost
CHECK (cost >= 0 AND cost <= 500);
```

Screenshot:

```
[db2 => ALTER TABLE Tours ADD CONSTRAINT check_tour_cost CHECK (cost >= 0 AND cost <= 500);
DB20000I The SQL command completed successfully.
```

Screenshot of attempt to insert violating record:

```
[db2 => INSERT INTO Tours (tour_id, description, cost, employee_id) VALUES (99, 'Invalid Tour', -1, 1);
DB21034E The command was processed as an SQL statement because it was not a valid Command Line Processor command. During SQL processing it returned:
SQL0545N The requested operation is not allowed because a row does not satisfy the check constraint "CS421G22.TOURS.CHECK_TOUR_COST". SQLSTATE=23513
```

Creativity

We choose to create a realistic dataset for the Artifacts table using actual artworks from the Louvre Museum. Each record represents a real artifact with accurate information about the piece's name, description, and date of creation. There are multiple tables that have a foreign key that references Artifacts, including Paintings, Sculpture, Photographs, Creates, Artifact_Genre, and Displays. All records in these tables were created to accurately reflect the artifact's category, associated artists, and genres. This approach provides a more realistic representation of how a museum organizes and stores information about its artworks.

The data was collected manually from the Louvre's official collections website (<https://collections.louvre.fr/>) and recorded in an Excel spreadsheet. A Python script was then used to convert the CSV file into SQL insert statements. The script goes through each row and formats the values correctly to ensure proper SQL syntax and to reduce typos when entering a large number of records manually.

```
tables = {
    {
        "input_file": "csv/Artifacts.csv",
        "output_file": "sql/artifacts_inserts.sql",
        "table_name": "Artifacts",
        "int_columns": [0, 3], # artifact_id, date_of_creation
        "date_columns": []
    },
}
```

```

for config in tables:
    input_file = config["input_file"]
    output_file = config["output_file"]
    table_name = config["table_name"]
    int_columns = config.get("int_columns", [])
    date_columns = config.get("date_columns", [])

    with open(input_file, newline='', encoding="utf-8") as csvfile:
        reader = csv.reader(csvfile)
        headers = [h.strip() for h in next(reader)] # remove extra spaces from header

        with open(output_file, "w", encoding="utf-8") as outfile:
            for row in reader:
                values = []
                for i, value in enumerate(row):
                    value = value.strip()
                    if value == "":
                        values.append("NULL")
                    elif i in int_columns:
                        values.append(str(int(value)))
                    elif i in date_columns:
                        # add quotes to dates and times
                        values.append(f"'{value}'")
                    else:
                        values.append("'" + value.replace("'", "''") + "'")

                sql = f"INSERT INTO {table_name} ({', '.join(headers)}) VALUES ({', '.join(values)});"
                outfile.write(sql + "\n")

    print(f"{output_file} created for {table_name}.")

```

Contributions

Our team held two meetings, during which we divided the tasks evenly and collectively reviewed the project deliverable and report prior to submission. Here are each member's contributions:

- Gonzalo: Tasks 5 and 8
- Annabelle: Task 3, database setup and password change
- Jessie: Tasks 4 and 9
- Clementine: Task 6 and 7

All team members contributed to proof reading.